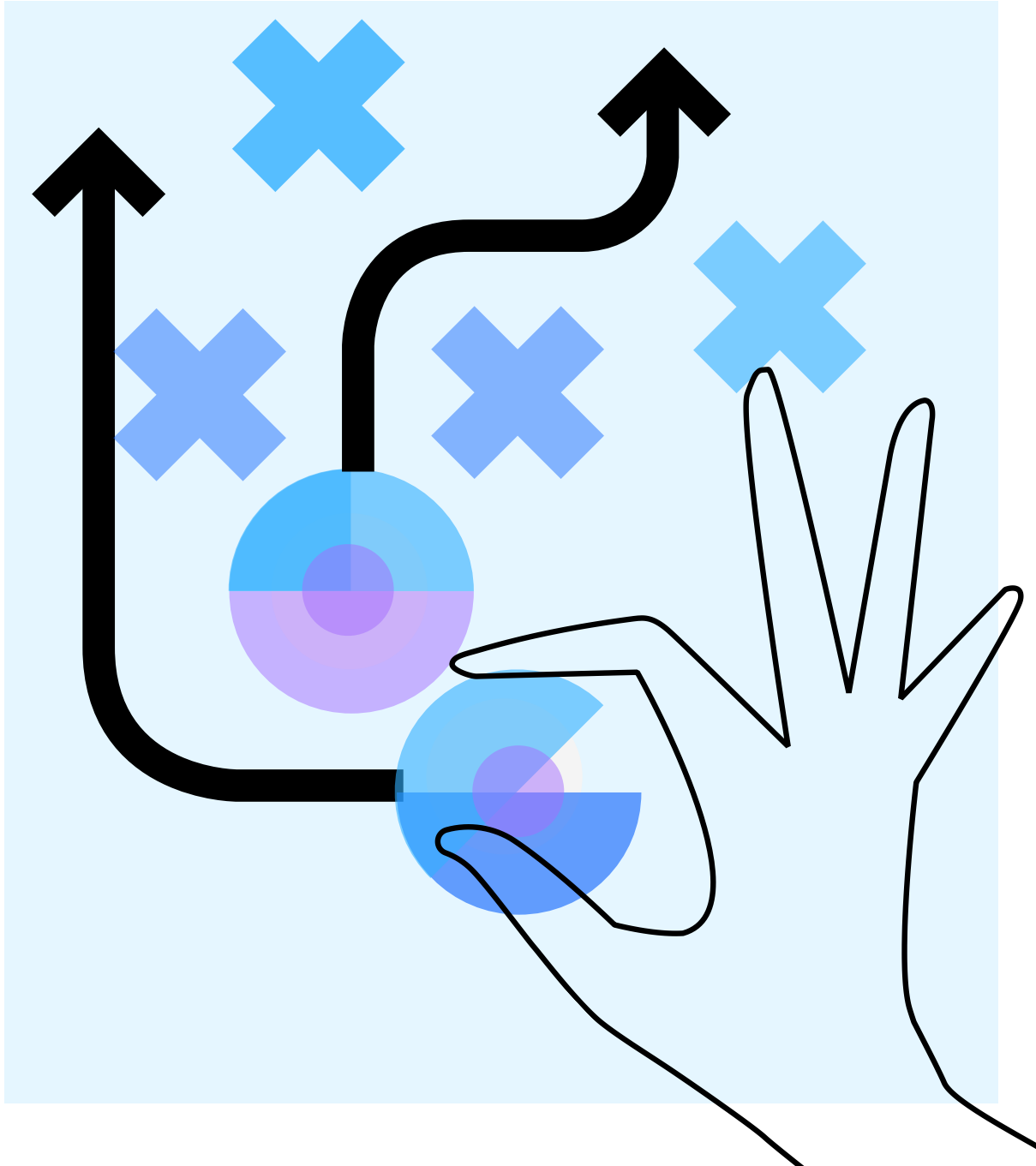
遠東金士頓

SuccessFactors Project

SAP BTP 服務生命週期管理 (Service Lifecycle)

IBM Consulting, Talent Transformation

2026.07.29



講師

課程表		課程內容備註
第一堂	預計授課日期：yyyymmdd	
	課程：BTP架構與核心概念 . BTP Global Account / Subaccount . Region / Environment . Entitlement . Service Plan . Multi-cloud architecture (AWS/Azure/GCP)	1. BTP是甚麼(Platform定位) 2. BTP四大能力 3. BTP架構 4. Account Model 概念介紹 5. BTP環境介紹 6. Extension Strategy
	熟悉 BTP Cockpit 畫面布局	
	學習材料、網站閱讀	
第二堂	預計授課日期：yyyymmdd	
	課程：BTP Account Model . Global Account . Directory . Subaccount . Quota/Entitlement	1. BTP帳號層級結構 2. 資源如何分配 3. 專案與系統如何隔離 4. 權限與管理方式
	建立Subaccount，分配Entitlement	
	學習材料、網站閱讀	
第三堂	預計授課日期：yyyymmdd	
	課程：BTP Environment . Cloud Foundry . Kyma . ABAP Environment . Neo(legacy)	1. BTP的Runtime Environment概念與架構 2. 各種Environment的用途與適用場景說明
	建立Cloud Foundry environment Org/Space	
	學習材料、網站閱讀	
第四堂	預計授課日期：yyyymmdd	
	課程：BTP Security . Identity Provider . Trust configuration . Role Collection . XSUAA	1. 介紹SAP BTP的安全架構與身份管理機制 2. 了解如何在雲端環境中管理使用者與權限
	建立 Role collection / SSO Login	
	學習材料、網站閱讀	
第五堂	預計授課日期：yyyymmdd	
	課程：BTP Connectivity . Destination service . Connectivity service . Cloud Connector	介紹 SAP BTP 的系統連線架構與技術機制，說明如何安全地讓 BTP 應用程式存取： . On-Premise SA統 . Cloud application . 外部 API
	建立Destination / 連接 On-Premise System	
	學習材料、網站閱讀	
第六堂	預計授課日期：yyyymmdd	
	課程：BTP Service Lifecycle . Service instance . Service binding . Service key	介紹 BTP 服務的生命週期管理（Service Lifecycle），說明如何： . 啟用服務 . 建立 Service Instance . 將服務綁定到應用程式 . 管理服務資源
	建立Service instance	
	學習材料、網站閱讀	

Topic

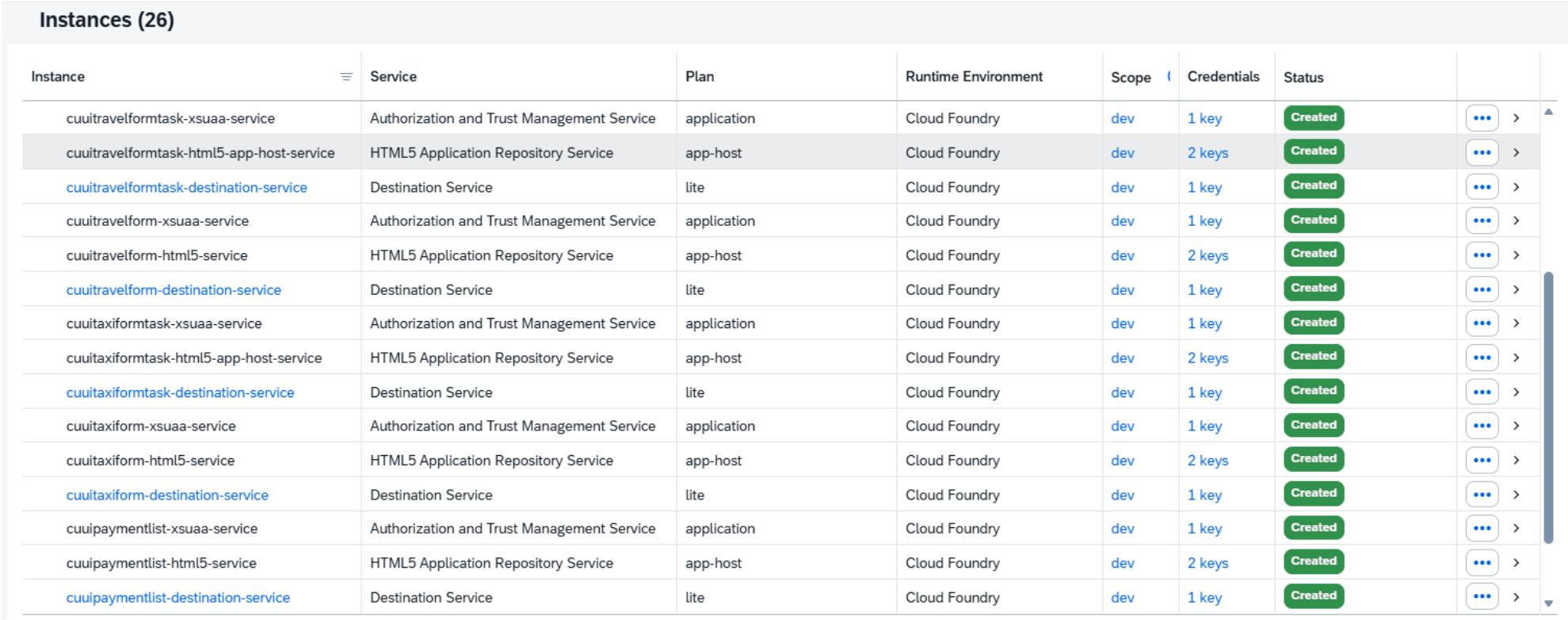
1. 核心概念
    - Service Instance
    - Service Binding
    - Service Key
2. 實務操作：生命週期四部曲
    - 步驟 1 & 2：從啟用至建立
    - 步驟 3 & 4：綁定與資源管理
3. 掌握 BTP 服務管理核心

核心概念（一）Service Instance

Service Instance (服務實例)

- **說明**：這是你在 BTP 上面向服務市集（Service Marketplace）實際部署或啟動的一個「服務物件」。它代表該服務的一份執行個體，例如一個資料庫或訊息佇列。

- **舉例**：在 BTP 建立一個 SAP HANA Cloud 的資料庫實例，或建立一個 PostgreSQL 資料庫服務。這時系統會為你準備好該資料庫的運行資源。



Instances (26)

Instance	Service	Plan	Runtime Environment	Scope	Credentials	Status
cuuitravelformtask-xsuaa-service	Authorization and Trust Management Service	application	Cloud Foundry	dev	1 key	Created
cuuitravelformtask-html5-app-host-service	HTML5 Application Repository Service	app-host	Cloud Foundry	dev	2 keys	Created
cuuitravelformtask-destination-service	Destination Service	lite	Cloud Foundry	dev	1 key	Created
cuuitravelform-xsuaa-service	Authorization and Trust Management Service	application	Cloud Foundry	dev	1 key	Created
cuuitravelform-html5-service	HTML5 Application Repository Service	app-host	Cloud Foundry	dev	2 keys	Created
cuuitravelform-destination-service	Destination Service	lite	Cloud Foundry	dev	1 key	Created
cuuitaxiformtask-xsuaa-service	Authorization and Trust Management Service	application	Cloud Foundry	dev	1 key	Created
cuuitaxiformtask-html5-app-host-service	HTML5 Application Repository Service	app-host	Cloud Foundry	dev	2 keys	Created
cuuitaxiformtask-destination-service	Destination Service	lite	Cloud Foundry	dev	1 key	Created
cuuitaxiform-xsuaa-service	Authorization and Trust Management Service	application	Cloud Foundry	dev	1 key	Created
cuuitaxiform-html5-service	HTML5 Application Repository Service	app-host	Cloud Foundry	dev	2 keys	Created
cuuitaxiform-destination-service	Destination Service	lite	Cloud Foundry	dev	1 key	Created
cuuipaymentlist-xsuaa-service	Authorization and Trust Management Service	application	Cloud Foundry	dev	1 key	Created
cuuipaymentlist-html5-service	HTML5 Application Repository Service	app-host	Cloud Foundry	dev	2 keys	Created
cuuipaymentlist-destination-service	Destination Service	lite	Cloud Foundry	dev	1 key	Created

核心概念（一）Service Instance

Instance Name	Description	
cuiitravelformtask	差旅表單任務相關	
cuiitravelform	差旅表單主程式相關	
cuiitaxiformtask	計程車/叫車表單任務相關	
cuiipaymentlist	付款清單相關	

後綴	Description	
-xsuaa-service	Authorization and Trust Management Serice ，負責安全認證與權限控管 (UAA)	
-html5-app-host-service	HTML5 Application Repository Service ， 負責託管前端 HTML5/SAPUI5 應用程式	
-destination-service	Destinations，負責配置與管理連線目標	
cuiipaymentlist	付款清單相關	

Instance	Service	Plan
cuuitravelformtask-xsuaa-service	Authorization and Trust Management Service	application
cuuitravelformtask-html5-app-host-service	HTML5 Application Repository Service	app-host
cuuitravelformtask-destination-service	Destination Service	lite
cuuitravelform-xsuaa-service	Authorization and Trust Management Service	application
cuuitravelform-html5-service	HTML5 Application Repository Service	app-host
cuuitravelform-destination-service	Destination Service	lite

核心概念（一）Service Instance

Instances (26)

Service instances created in: Cloud Foundry | Kyma/Kubernetes | Other environments

Instance	Service	Plan	Runtime Environment	Scope	Credentials
sap_work_zone	SAP Build Work Zone, standard edition	standard	Cloud Foundry	dev	
sap_process_automation	SAP Build Process Automation	standard	Cloud Foundry	dev	1 key
ordersmanagement-xsuaa-service	Authorization and Trust Management Service	application	Cloud Foundry	dev	1 key
ordersmanagement-html5-app-host-service	HTML5 Application Repository Service	app-host	Cloud Foundry	dev	2 keys
ordersmanagement-destination-service	Destination Service	lite	Cloud Foundry	dev	1 key
hana_multitenancy_admin	SAP HANA Cloud	hana-multitenancy-admin	Cloud Foundry	dev	
hana_cloud_dev	SAP HANA Cloud	hana	Cloud Foundry	dev	
hana_admin_api_access	SAP HANA Cloud	admin-api-access	Cloud Foundry	dev	

核心概念（二）Service Binding

定義說明：

- **將服務實例連接到應用程式的橋樑。**
- **自動將服務的存取憑證（如 URL、帳號、密碼）注入到指定的應用程式環境變數中。**

為什麼需要 Service Binding？

在傳統開發中，如果程式（App）要連接資料庫，工程師通常會把資料庫的 `IP 位址、帳號、密碼` 直接寫死在程式碼的設定檔（如 `config.json` 或 `application.properties`）裡。

但在雲端環境中，這樣做有兩大致命缺點：

1.不安全：密碼寫在程式碼裡，一旦程式碼上傳到 GitHub，全世界都會看到你的密碼。

2.不彈性：如果資料庫今天換了新的伺服器、IP 變了，工程師就必須修改程式碼、重新編譯、重新部署，非常麻煩。

Service Binding 就是為了解決這個痛點而生的「隔離層」。
當你執行 Binding（綁定）時：

•雲端平台會自動產生一組**專屬的存取憑證**（Credentials）。

•平台會把這組憑證，像打針一樣，直接「注入」到應用程式運行的**環境變數（Environment Variables）**中（在 Cloud Foundry 中通常叫做 **`VCAP_SERVICES`**）。

•你的應用程式只需要去讀取環境變數，不需要知道真實的密碼是什麼。

核心概念（二）Service Binding

假設您用 Node.js 寫了一個**訂單管理應用程式 (ordersmanagement)**，現在需要讀取 **SAP HANA 資料庫 (hana_cloud_dev)**。

1. 沒做 Service Binding 之前：

應用程式雖然部署上雲端了，但它是一座孤島。它不知道資料庫在哪裡，如果此時使用者開啟網頁，畫面會報錯（例如 `500 Connection Refused`）。

2. 進行 Service Binding（在 BTP 控制台點擊「Bind to Application」）：

當你把 `ordersmanagement`（應用程式）與 `hana_cloud_dev`（服務實例）綁定在一起時，系統底層會自動把以下這串 JSON 格式的憑證注入到應用程式的環境中：

```json
{
  "hana": [
    {
      "name": "hana_cloud_dev",
      "credentials": {
        "host": "://ondemand.com",
        "port": "443",
        "user": "DBADMIN_USER_ABC",
        "password": "SuperSecretPassword123!"
      }
    }
  ]
}
```

3. 綁定後的結果：

工程師寫代碼時，只需要寫：`db.connect(process.env.VCAP_SERVICES.hana)`。

- **安全性提高**：原始碼中完全看不到資料庫密碼。
- **運維極度方便**：如果這台 HANA Cloud 資料庫進行了升級、密碼變更或更換了網址，BTP 平台會**自動更新**注入進去的憑證，您的應用程式完全不需修改任何一行代碼，依然能正常運作。

核心概念（三）Service Key

什麼是 Service Key？

定義說明：

- **用於外部系統或本地開發環境存取服務的憑證。**
- **無需將服務綁定到特定應用程式，即可直接產生一組 JSON 格式的憑證資料。**

日常比喻：就像一把「備用鑰匙」，讓外面的合作夥伴或本地調試工具也能開門進入。

在 Service Binding 的概念中，憑證是被自動「注入」到雲端平台（如 Cloud Foundry）的環境變數中。這意味著，**只有同樣住在這個雲端平台上的應用程式，才能享受這種自動注入的便利。**

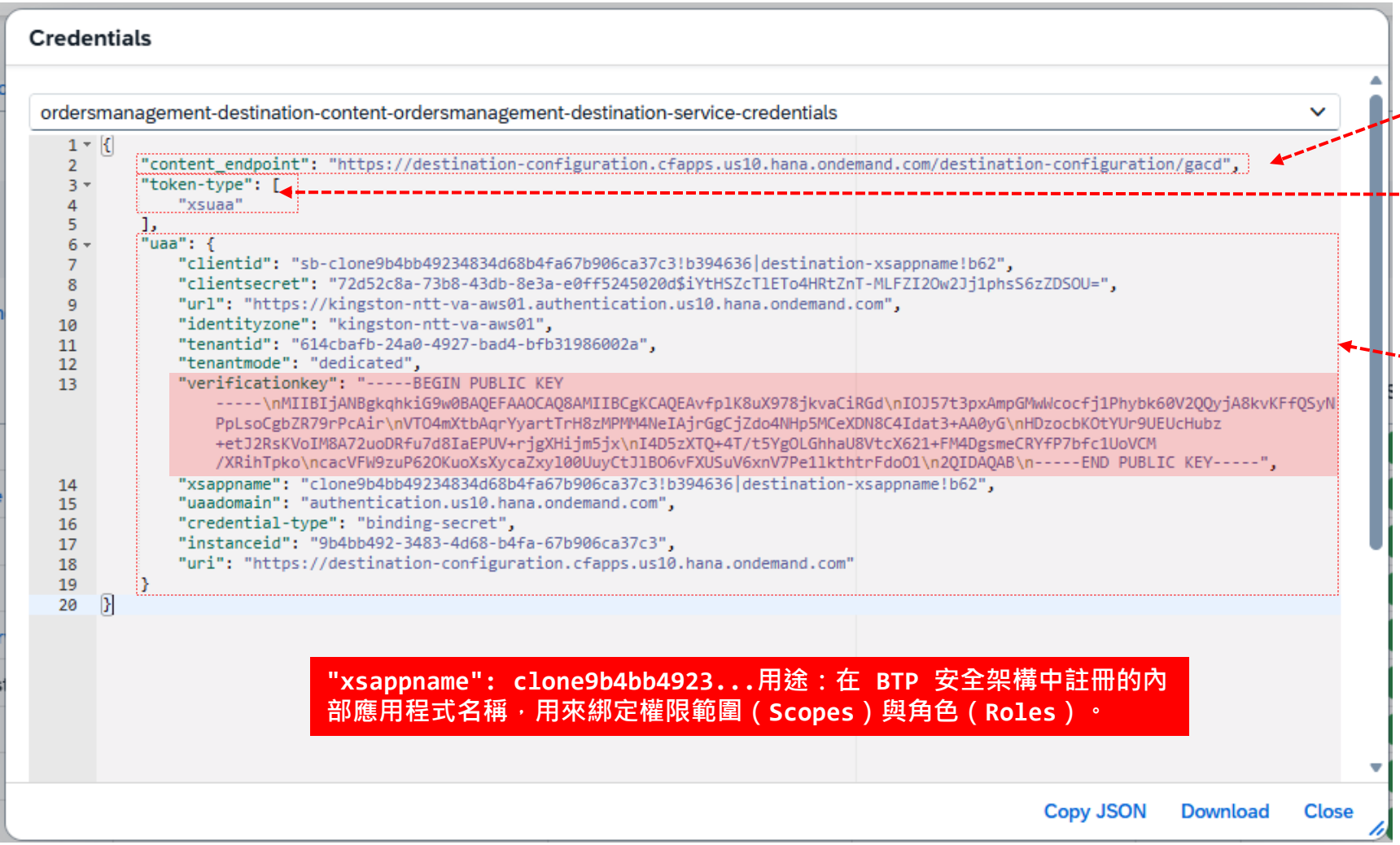
但在現實開發中，常常會遇到以下兩種情況：

1.本地開發與除錯 (Local Development)：工程師在自己的筆記型電腦上寫代碼（這時程式還沒部署到 BTP 雲端），電腦要怎麼連上雲端的 SAP HANA 資料庫或 Destination 服務？

2.外部系統整合 (Third-party Integration)：公司的其他獨立系統（例如地端的 ERP、微軟的 Power BI、或者是外部合作廠商的系統），需要呼叫 BTP 上的某個服務。

這時候，外面的人進不來，我們也無法對外「注入」環境變數。
Service Key 就是為了解決這個問題而生的「實體鑰匙」。它是一個你可以直接**複製、貼上或下載**的 JSON 檔案，裡面寫滿了連線所需的所有網址與密鑰。

核心概念（三）Service Key (BTP Destination Service)



實際 API 網址

使用 SAP 的 XSUAA 安全架構來進行憑證（Token）管理

UAA 安全驗證區塊 (身分驗證與授權)

加密公開金鑰（Public Key）。當您的應用程式收到驗證伺服器發給您的通行證（Token）時，程式可以使用這串公開金鑰來自我驗證該 Token 是否真的是由 SAP 官方核發、有沒有被中途篡改。

實務操作：生命週期四部曲

流程圖設計：建議使用橫向的 4 步驟箭頭流程圖：

1. **啟用服務** (Enable)
2. **建立實例** (Create Instance)
3. **應用綁定** (Bind Application)
4. **資源管理** (Manage Resources)

步驟 1 & 2：從啟用至建立

服務的誕生

啟用服務：

1. 進入 BTP Cockpit 帳戶。
2. 在 Service Marketplace (服務市場) 瀏覽並選擇所需服務。

建立 Service Instance：

1. 選擇對應的服務方案 (Plan)。
2. 填寫實例名稱並完成建立。

步驟 3 & 4：綁定與資源管理

服務的應用與維護

將服務綁定到應用程式：

1. 在應用程式中宣告依賴。
2. 執行 Binding 操作，使應用程式獲得存取權限。

管理服務資源：

1. 監控服務使用量與效能。
2. 根據需求升級方案、更新設定或刪除過期資源。

總結 -- 掌握 BTP 服務管理核心

重點回顧：

1. Instance 是資源實體。
2. Binding 是內部連接。
3. Key 是外部憑證。

最佳實踐：嚴格管理 Service Key 的權限，並定期清理未使用的 Service Instance 以節省成本。

Q&A